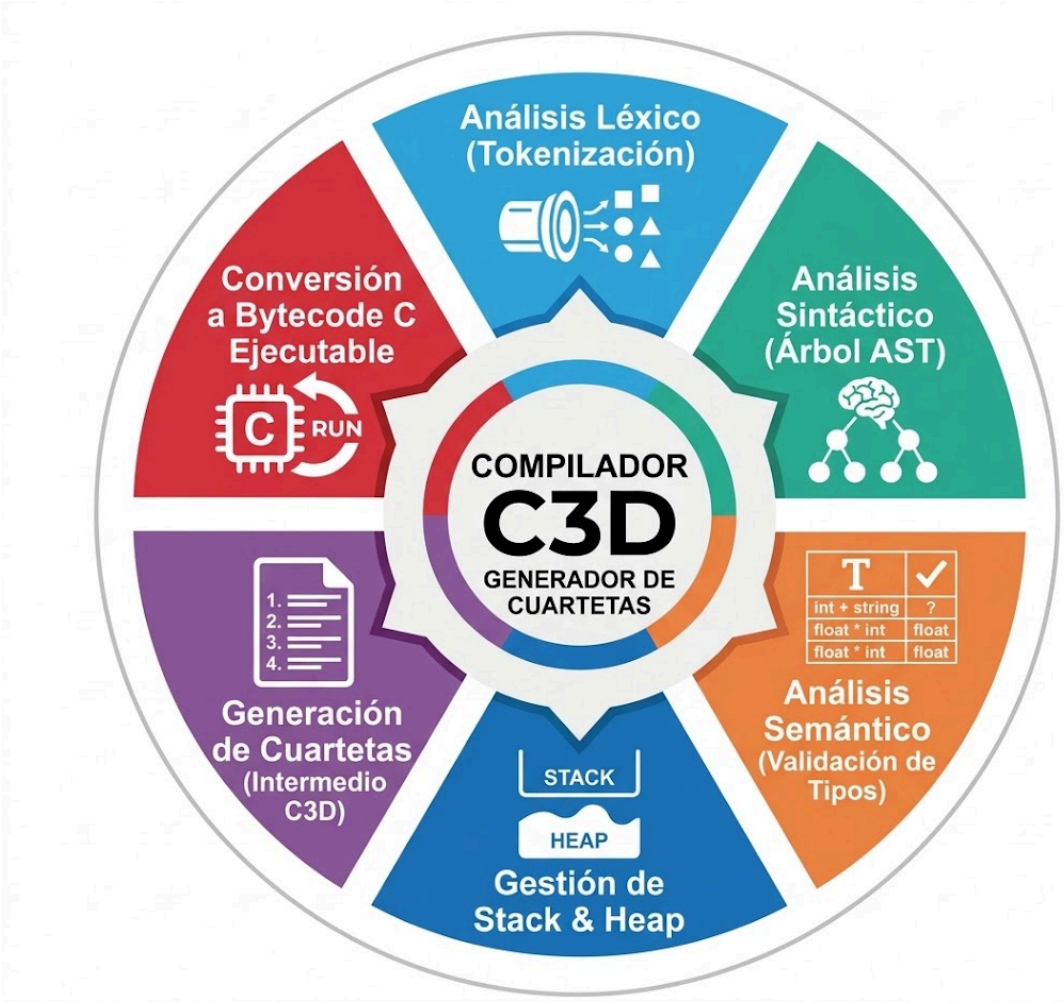
MANUAL DE USUARIO

IDE XS2



Javier Alejandro Mérida Gómez 202230638

27/10/2026

ÍNDICE

DESCRIPCIÓN GENERAL **3**
CARACTERÍSTICAS DESTACADAS **3**
EJECUCIÓN DEL SISTEMA **4**
FUNCIONAMIENTO DE LA APLICACIÓN **4**
1. ARCHIVOS 4
2. EDITOR DE TEXTO Y ÁRBOL DE TRABAJO 5
3. SECCIÓN DE ERRORES 7
4. MENÚ GENERADO 8
a. TABLAS DE SÍMBOLOS 8
b. TABLA DE TIPOS 9
c. TABLA DE COMPATIBILIDAD 9
d. CUARTETAS 10
e. CODIGO 3D 11
5. CÓDIGO RISC-V 11

DESCRIPCIÓN GENERAL

IDE XS2 es un entorno de desarrollo para tres lenguajes de alto nivel: Pig Latin, Y? y Zetariano. Valida cada programa de forma léxica, sintáctica y semántica, lo traduce a código de tres direcciones por medio de cuartetas y, a partir de ellas, genera código ensamblador RISC-V funcional.

Durante la compilación se pueden consultar la tabla de símbolos, la tabla de tipos, la tabla de compatibilidad, las cuartetas, el código de tres direcciones y el código RISC-V generado.

CARACTERÍSTICAS DESTACADAS

- Análisis léxico, sintáctico y semántico de Pig Latin, Y? y Zetariano.
- Programas de varios archivos: el .pig principal importa archivos .y y .z.
- Programación orientada a objetos en Zetariano
- Coloreado del código en tiempo real para los tres lenguajes.
- Tabla de símbolos, de tipos y de compatibilidad.
- Generación de código de tres direcciones y cuartetas usando solo el stack y el heap.
- Traducción a ensamblador RISC-V funcional, listo para ejecutarse en RARS.
- Tabulación automática del código.
- Manejo de proyectos, carpetas y archivos: crear, abrir, guardar, eliminar y descargar en .zip.

EJECUCIÓN DEL SISTEMA

Una vez descargada la aplicación, para iniciarla es necesario ejecutar el archivo .jar desde la línea de comandos (se necesita Java 21 o superior). Sigue estos pasos:

1. Abre una terminal en la ubicación donde se encuentra el archivo .jar.
2. Ejecuta el siguiente comando:

   ```
   java -jar ProyectoDosCompi2-1.0-SNAPSHOT.jar
   ```

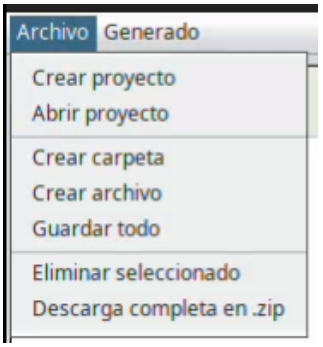
3. La aplicación se abrirá y quedará lista para usarse.

FUNCIONAMIENTO DE LA APLICACIÓN

1. ARCHIVOS

Se cuenta con un amplio menú para el manejo de archivos, con las siguientes opciones:

- **Crear proyecto:** crea la carpeta del proyecto y la abre en el árbol de trabajo.
- **Abrir proyecto:** abre una carpeta existente como proyecto.
- **Crear carpeta:** crea una carpeta dentro del proyecto.
- **Crear archivo:** crea un archivo .pig, .y o .z; sin proyecto abierto, abre una pestaña nueva.
- **Guardar todo:** guarda todas las pestañas abiertas.
- **Eliminar seleccionado:** elimina el archivo o la carpeta seleccionada en el árbol.
- **Descarga completa en .zip:** descarga todo el proyecto en un archivo comprimido.



2. EDITOR DE TEXTO Y ÁRBOL DE TRABAJO

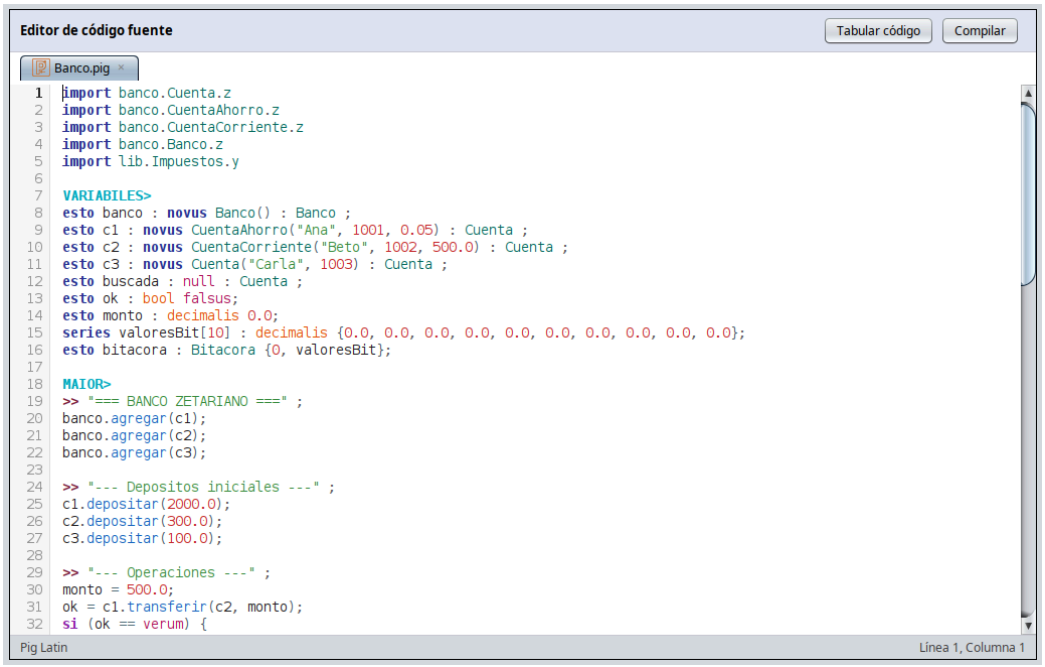
Para el editor de texto y árbol de trabajo se divide en dos secciones diferentes, que se detallan a continuación.

- Editor de texto
    - El editor de texto soporta varias pestañas de diferentes tipos de archivos.
    - El código se colorea en tiempo real según el lenguaje de cada archivo, sin cursivas ni fondos, con los colores de la siguiente tabla.

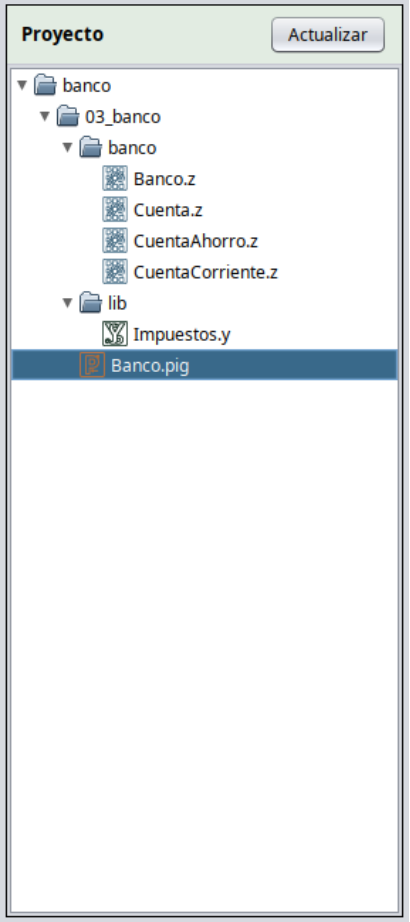
Categoría	Color
Control de flujo (si, dum, per, if, while, switch, mientras, elegir, retornar…)	Violeta #8E24AA, negrita
Declaraciones y modificadores (import, esto, series, novus, class, public, extends, new, definir, estructura)	Índigo #283593, negrita
Tipos de dato (numerus, decimalis, int, double, String, entero, flotante, void…)	Naranja #E65100
Valores especiales (verum, falsus, true, false, verdadero, falso, null, this, super)	Rosa #AD1457
Anotación @Override	Dorado #9E7C00, negrita
Entrada y salida (>>, <<, print, println, readln, imprimir, leer)	Corinto #7B1E3A, negrita
Secciones (VARIABILES>, MAIOR>, FINIS, %estructuras, %funciones)	Cian #00ACC1, negrita
Nombres de clases y estructuras	Verde azulado #00695C
Funciones y métodos	Azul #1565C0
Números	Rojo #C62828
Cadenas y caracteres	Verde #388E3C
Comentarios	Gris #8A8A8A
Operadores	Gris azulado #546E7A

Signos de agrupación ( ) [ ] { }	Gris pizarra #455A64
Puntuación ; , . :	Gris #616161
Identificadores	Casi negro #1E1E1E
Errores léxicos	Rojo #D50000, negrita y subrayado

- Cuenta con conteo de líneas y columnas para una mejor accesibilidad y uso del editor de texto.
- El botón “Tabular código” ordena la sangría del archivo abierto: por llaves en Pig Latin y Zetariano, y por niveles de indentación en Y?.
- El botón “Compilar”, a la par de Tabular, compila el programa. Con un proyecto abierto compila su archivo .pig junto con los archivos que importa (si hay varios .pig pregunta cuál); sin proyecto compila el archivo de la pestaña actual.
- Cada pestaña muestra el nombre de su archivo y se cierra con su botón x.



- Árbol de trabajo
    - Dentro de esta sección vamos a visualizar la raíz de nuestro proyecto y ver las carpetas o archivos que tenemos.
    - Por cada extensión de archivo contamos con un icono que los identifica.

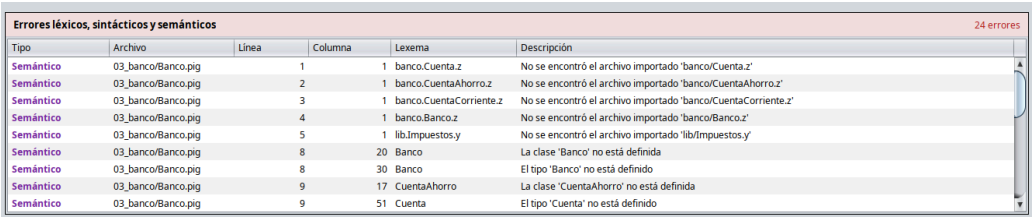


3. SECCIÓN DE ERRORES

En esta sección se listan los errores léxicos, sintácticos y semánticos de todos los archivos del programa: el .pig principal y los archivos .y y .z que importa. Arriba se muestra el total de errores (o “Sin errores”) y la columna Tipo se colorea según la clase de error. Para cada error se muestra:

- Tipo
- Archivo
- Línea
- Columna
- Lexema

- Descripción



Errores léxicos, sintácticos y semánticos — 24 errores

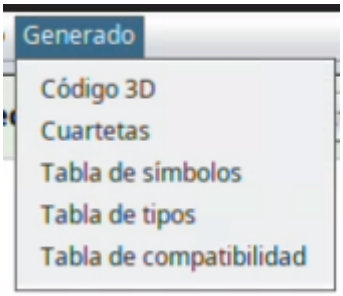
Tipo	Archivo	Línea	Columna	Lexema	Descripción
Semántico	03_banco/Banco.pig	1	1	banco.Cuenta.z	No se encontró el archivo importado 'banco/Cuenta.z'
Semántico	03_banco/Banco.pig	2	1	banco.CuentaAhorro.z	No se encontró el archivo importado 'banco/CuentaAhorro.z'
Semántico	03_banco/Banco.pig	3	1	banco.CuentaCorriente.z	No se encontró el archivo importado 'banco/CuentaCorriente.z'
Semántico	03_banco/Banco.pig	4	1	banco.Banco.z	No se encontró el archivo importado 'banco/Banco.z'
Semántico	03_banco/Banco.pig	5	1	lib.Impuestos.y	No se encontró el archivo importado 'lib/Impuestos.y'
Semántico	03_banco/Banco.pig	8	20	Banco	La clase 'Banco' no está definida
Semántico	03_banco/Banco.pig	8	30	Banco	El tipo 'Banco' no está definido
Semántico	03_banco/Banco.pig	9	17	CuentaAhorro	La clase 'CuentaAhorro' no está definida
Semántico	03_banco/Banco.pig	9	51	Cuenta	El tipo 'Cuenta' no está definido

4. MENÚ GENERADO

En el menú Generado se encuentran los elementos que produce la compilación. El código 3D y las cuartetas solo se generan cuando el programa no tiene errores; las tablas se llenan aunque haya errores.

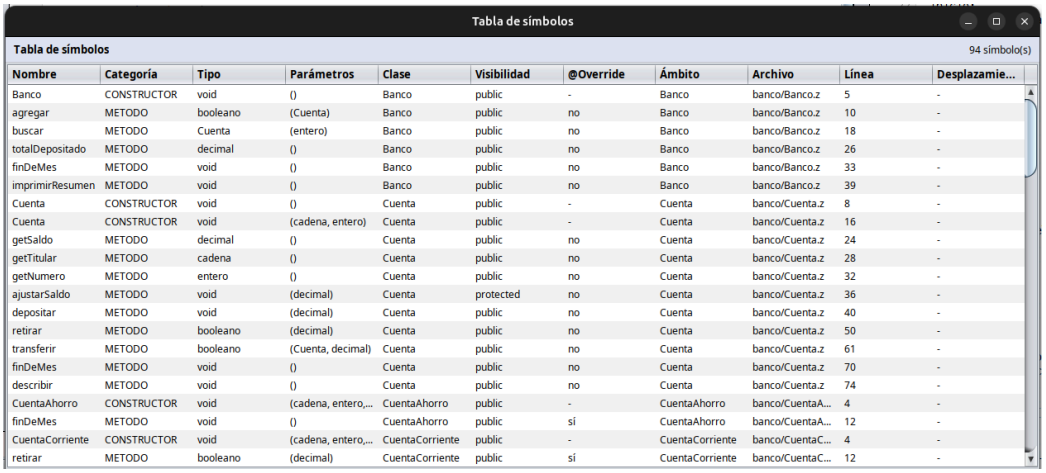
En ellos encontraremos:

- Código 3D
- Cuartetas
- Tabla de símbolos
- Tabla de tipos
- Tabla de compatibilidad



a. TABLAS DE SÍMBOLOS

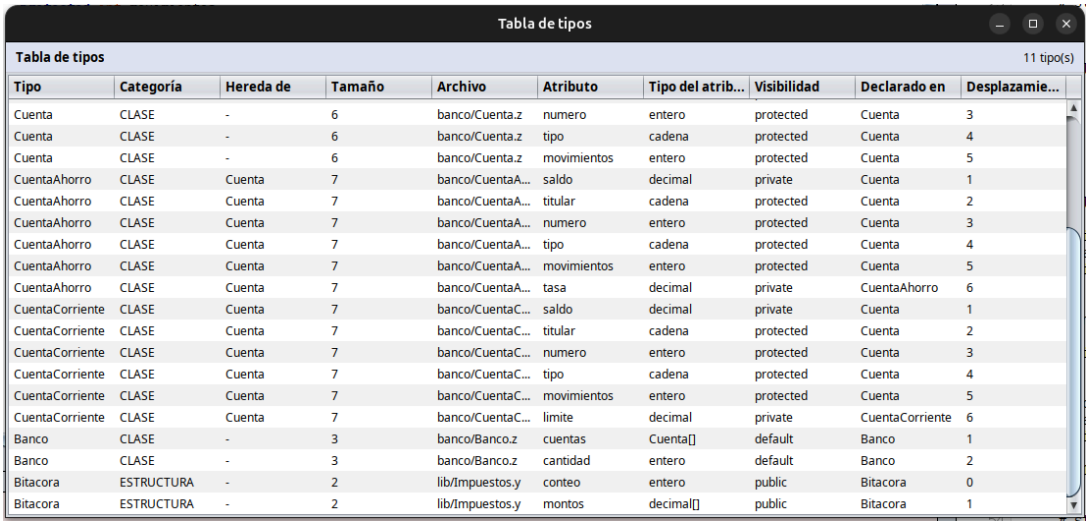
Muestra las variables, parámetros, funciones, métodos y constructores del programa con su categoría, tipo, parámetros, clase, visibilidad, @Override, ámbito, archivo, línea y desplazamiento en el marco.



Nombre	Categoría	Tipo	Parámetros	Clase	Visibilidad	@Override	Ámbito	Archivo	Línea	Desplazamie...
Banco	CONSTRUCTOR	void	()	Banco	public	-	Banco	banco/Banco.z	5	-
agregar	METODO	booleano	(Cuenta)	Banco	public	no	Banco	banco/Banco.z	10	-
buscar	METODO	Cuenta	(entero)	Banco	public	no	Banco	banco/Banco.z	18	-
totalDepositado	METODO	decimal	()	Banco	public	no	Banco	banco/Banco.z	26	-
finDeMes	METODO	void	()	Banco	public	no	Banco	banco/Banco.z	33	-
imprimirResumen	METODO	void	()	Banco	public	no	Banco	banco/Banco.z	39	-
Cuenta	CONSTRUCTOR	void	()	Cuenta	public	-	Cuenta	banco/Cuenta.z	8	-
Cuenta	CONSTRUCTOR	void	(cadena, entero)	Cuenta	public	-	Cuenta	banco/Cuenta.z	16	-
getSaldo	METODO	decimal	()	Cuenta	public	no	Cuenta	banco/Cuenta.z	24	-
getTitular	METODO	cadena	()	Cuenta	public	no	Cuenta	banco/Cuenta.z	28	-
getNumero	METODO	entero	()	Cuenta	public	no	Cuenta	banco/Cuenta.z	32	-
ajustarSaldo	METODO	void	(decimal)	Cuenta	protected	no	Cuenta	banco/Cuenta.z	36	-
depositar	METODO	void	(decimal)	Cuenta	public	no	Cuenta	banco/Cuenta.z	40	-
retirar	METODO	booleano	(decimal)	Cuenta	public	no	Cuenta	banco/Cuenta.z	50	-
transferir	METODO	booleano	(Cuenta, decimal)	Cuenta	public	no	Cuenta	banco/Cuenta.z	61	-
finDeMes	METODO	void	()	Cuenta	public	no	Cuenta	banco/Cuenta.z	70	-
describir	METODO	void	()	Cuenta	public	no	Cuenta	banco/Cuenta.z	74	-
CuentaAhorro	CONSTRUCTOR	void	(cadena, entero,...	CuentaAhorro	public	-	CuentaAhorro	banco/CuentaA...	4	-
finDeMes	METODO	void	()	CuentaAhorro	public	si	CuentaAhorro	banco/CuentaA...	12	-
CuentaCorriente	CONSTRUCTOR	void	(cadena, entero,...	CuentaCorriente	public	-	CuentaCorriente	banco/CuentaC...	4	-
retirar	METODO	booleano	(decimal)	CuentaCorriente	public	si	CuentaCorriente	banco/CuentaC...	12	-

b. TABLA DE TIPOS

Muestra las estructuras de Y? y las clases de Zetariano con su categoría, la clase de la que heredan, su tamaño y su archivo, y cada atributo con su tipo, visibilidad, clase donde se declaró y desplazamiento.



Tipo	Categoría	Hereda de	Tamaño	Archivo	Atributo	Tipo del atrib...	Visibilidad	Declarado en	Desplazamie...
Cuenta	CLASE	-	6	banco/Cuenta.z	numero	entero	protected	Cuenta	3
Cuenta	CLASE	-	6	banco/Cuenta.z	tipo	cadena	protected	Cuenta	4
Cuenta	CLASE	-	6	banco/Cuenta.z	movimientos	entero	protected	Cuenta	5
CuentaAhorro	CLASE	Cuenta	7	banco/CuentaA...	saldo	decimal	private	Cuenta	1
CuentaAhorro	CLASE	Cuenta	7	banco/CuentaA...	titular	cadena	protected	Cuenta	2
CuentaAhorro	CLASE	Cuenta	7	banco/CuentaA...	numero	entero	protected	Cuenta	3
CuentaAhorro	CLASE	Cuenta	7	banco/CuentaA...	tipo	cadena	protected	Cuenta	4
CuentaAhorro	CLASE	Cuenta	7	banco/CuentaA...	movimientos	entero	protected	Cuenta	5
CuentaAhorro	CLASE	Cuenta	7	banco/CuentaA...	tasa	decimal	private	CuentaAhorro	6
CuentaCorriente	CLASE	Cuenta	7	banco/CuentaC...	saldo	decimal	private	Cuenta	1
CuentaCorriente	CLASE	Cuenta	7	banco/CuentaC...	titular	cadena	protected	Cuenta	2
CuentaCorriente	CLASE	Cuenta	7	banco/CuentaC...	numero	entero	protected	Cuenta	3
CuentaCorriente	CLASE	Cuenta	7	banco/CuentaC...	tipo	cadena	protected	Cuenta	4
CuentaCorriente	CLASE	Cuenta	7	banco/CuentaC...	movimientos	entero	protected	Cuenta	5
CuentaCorriente	CLASE	Cuenta	7	banco/CuentaC...	limite	decimal	private	CuentaCorriente	6
Banco	CLASE	-	3	banco/Banco.z	cuentas	Cuenta[]	default	Banco	1
Banco	CLASE	-	3	banco/Banco.z	cantidad	entero	default	Banco	2
Bitacora	ESTRUCTURA	-	2	lib/Impuestos.y	conteo	entero	public	Bitacora	0
Bitacora	ESTRUCTURA	-	2	lib/Impuestos.y	montos	decimal[]	public	Bitacora	1

c. TABLA DE COMPATIBILIDAD

Muestra, para el lenguaje que se elija, el tipo que resulta de cada operación entre dos tipos (error si no se permite) y si una variable de un tipo puede recibir un valor de otro.

Tabla de compatibilidad de tipos

Tabla de compatibilidad de tipos Lenguaje: Pig Latin

Cada celda es el tipo del resultado de *izquierdo operador derecho* (**error** si no se permite). **Asignar**: si una variable del tipo izquierdo puede recibir un valor del tipo derecho. Las reglas son las mismas en los tres lenguajes; solo cambian los nombres de los tipos. Un objeto se puede asignar a una variable de su clase o de un ancestro, y null a cualquier objeto o arreglo.

Izquierdo	Derecho	+	- * /	%	< > <= >=	== !=	&& ||	Asignar
numerus	numerus	numerus	numerus	numerus	bool	bool	error	sí
numerus	decimalis	decimalis	decimalis	error	bool	bool	error	error
numerus	textum	textum	error	error	error	error	error	error
numerus	littera	numerus	numerus	numerus	bool	bool	error	sí (implícita)
numerus	bool	error	error	error	error	error	error	error
decimalis	numerus	decimalis	decimalis	error	bool	bool	error	sí (implícita)
decimalis	decimalis	decimalis	decimalis	error	bool	bool	error	sí
decimalis	textum	textum	error	error	error	error	error	error
decimalis	littera	decimalis	decimalis	error	bool	bool	error	sí (implícita)
decimalis	bool	error	error	error	error	error	error	error
textum	numerus	textum	error	error	error	error	error	error
textum	decimalis	textum	error	error	error	error	error	error
textum	textum	textum	error	error	error	bool	error	sí
textum	littera	textum	error	error	error	error	error	error
textum	bool	textum	error	error	error	error	error	error
littera	numerus	numerus	numerus	numerus	bool	bool	error	error
littera	decimalis	decimalis	decimalis	error	bool	bool	error	error
littera	textum	textum	error	error	error	error	error	error
littera	littera	numerus	numerus	numerus	bool	bool	error	sí
littera	bool	error	error	error	error	error	error	error
bool	numerus	error	error	error	error	error	error	error
bool	decimalis	error	error	error	error	error	error	error
bool	textum	textum	error	error	error	error	error	error
bool	littera	error	error	error	error	error	error	error
bool	bool	error	error	error	error	bool	bool	sí

Un solo operando: **-x**: numerus → numerus, decimalis → decimalis, littera → numerus | **!x / non x**: bool → bool | **++ --**: numerus → numerus, decimalis → decimalis, littera → littera

d. CUARTETAS

Muestra las cuartetas enumeradas en el orden en que se generaron, con su operador, argumento 1, argumento 2 y resultado. A partir de ellas se producen el código de tres direcciones y el código RISC-V.

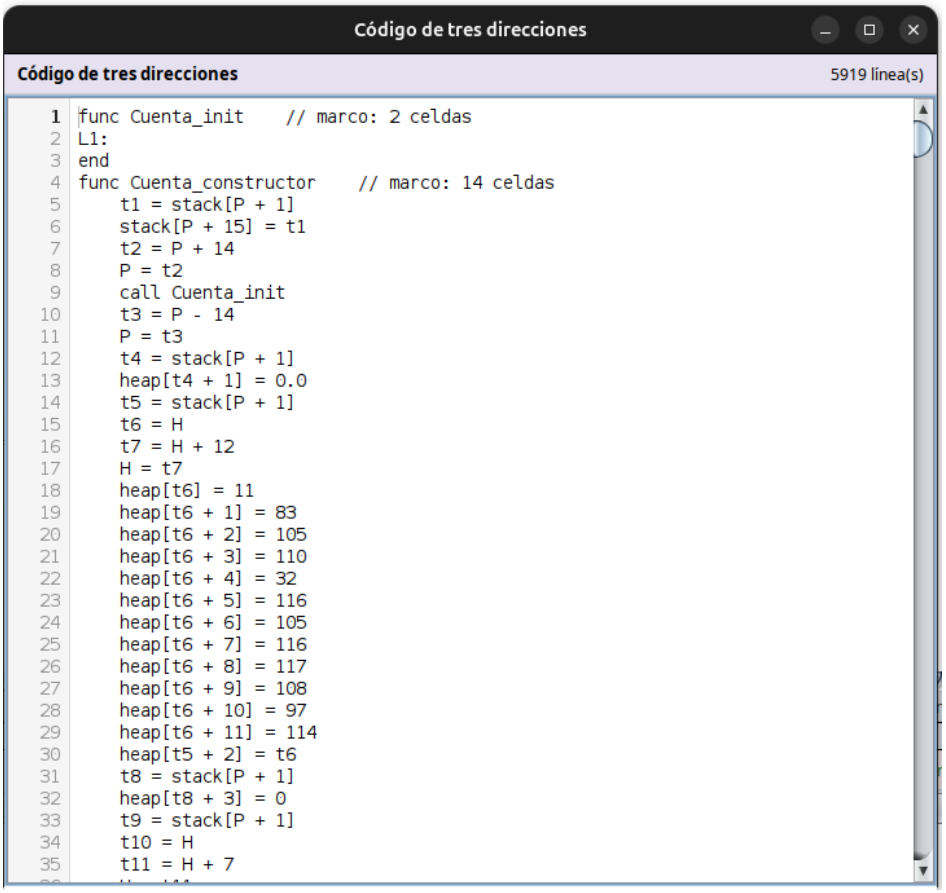
Cuartetas

Cuartetas 5919 cuarteta(s)

#	Operador	Argumento 1	Argumento 2	Resultado
1	func	2		Cuenta_init
2	label			L1
3	end			
4	func	14		Cuenta_constructor
5	leer_stack	P	1	t1
6	escribir_stack	t1	15	P
7	+	P	14	t2
8	=	t2		P
9	call			Cuenta_init
10	-	P	14	t3
11	=	t3		P
12	leer_stack	P	1	t4
13	escribir_heap	0.0	1	t4
14	leer_stack	P	1	t5
15	=	H		t6
16	+	H	12	t7
17	=	t7		H
18	escribir_heap	11	0	t6
19	escribir_heap	83	1	t6
20	escribir_heap	105	2	t6
21	escribir_heap	110	3	t6
22	escribir_heap	32	4	t6
23	escribir_heap	116	5	t6
24	escribir_heap	105	6	t6

e. CODIGO 3D

Muestra el código de tres direcciones del programa: asignaciones con temporales, saltos con etiquetas y lecturas y escrituras en el stack y el heap. P es el puntero del marco de la función actual y H la siguiente celda libre del heap.



5. CÓDIGO RISC-V

Cuando el programa compila sin errores, el panel inferior muestra el código ensamblador RISC-V generado a partir de las cuartetas, con coloreado y números de línea. Sobre cada grupo de instrucciones aparece como comentario la instrucción de tres direcciones que traduce.

Con el botón Guardar el código se guarda como archivo .s o .asm, y con Copiar se copia al portapapeles. El archivo se ejecuta en el simulador RARS: se abre el archivo, se ensambla con Assemble y se ejecuta con Run. Lo que pide el programa se escribe en la ventana Run I/O.

Código ensamblador RISC-V

```
_decimal9:      .double 300.0
_decimal10:     .double 1200.0
_decimal11:     .double 200.0
_decimal12:     .double 150.0
_decimal13:     .double 50.0

.text
inicio:
    # stack y heap del codigo de tres direcciones
    li        s0, 2143289344
    li        a0, 3145728
    li        a7, 9
    ecall
    mv        s1, a0
    li        s2, 0
    li        s3, 0
    mv        s4, s0
    li        s5, 0
    jal       ra, main
    li        a7, 10
    ecall

Cuenta_init:
    # marco: 2 celdas
    addi      sp, sp, -4
    sw        ra, 0(sp)
L1:
    # end
    lw        ra, 0(sp)
    addi      sp, sp, 4
    ret

Cuenta_constructor:
    # marco: 14 celdas
    addi      sp, sp, -4
    sw        ra, 0(sp)
    # t1 = stack[P + 1]
    lw        t2, 8(s4)
    sw        t2, 16(s4)
    # stack[P + 15] = t1
    lw        t2, 16(s4)
    sw        t2, 120(s4)
    # t2 = P + 14
    mv        t0, s2
    li        t1, 14
    add       t2, t0, t1
    sw        t2, 24(s4)
    # P = t2
    lw        t0, 24(s4)
    mv        s2, t0
```

27206 líneas

Guardar | Compilar | Copiar